Enforcing Mandatory Fields Using UI Policies and Migrating Changes with Update Sets

Introduction:

- In modern IT service management, data accuracy and consistency are critical.
- ServiceNow provides powerful tools to enforce form behavior and manage changes.

This project focuses on:

- Enforcing mandatory fields dynamically
- Migrating configurations between instances

1. Abstract

This project focuses on implementing dynamic form behavior and efficient change migration in ServiceNow. UI Policies are used to enforce mandatory fields based on specific conditions, ensuring data accuracy and consistency. Additionally, Update Sets are utilized to track and migrate configuration changes between different environments. The project demonstrates how organizations can maintain data integrity while ensuring smooth deployment of configurations across instances.

2. Objective

The main objectives of this project are:

- To understand and implement UI Policies in ServiceNow.
- To enforce mandatory fields dynamically based on conditions.
- To learn how to create and manage Update Sets.
- To migrate configurations from one instance to another using Update Sets.
- To test and validate changes effectively in the Incident module.

What are UI Policies?

- UI Policies are rules that control form behavior dynamically
- They run on the client side
- No scripting required (optional scripting supported)

Key Uses:

- Make fields mandatory
- Make fields read-only
- Show/Hide fields

3. Technology Description

ServiceNow Platform

ServiceNow is a cloud-based platform used for IT Service Management (ITSM). It provides tools to automate workflows, manage incidents, and configure system behavior.

UI Policy

UI Policies are used to control the behavior of fields on forms without scripting. They help in:

- Making fields mandatory
- Making fields read-only
- Showing or hiding fields

Update Sets

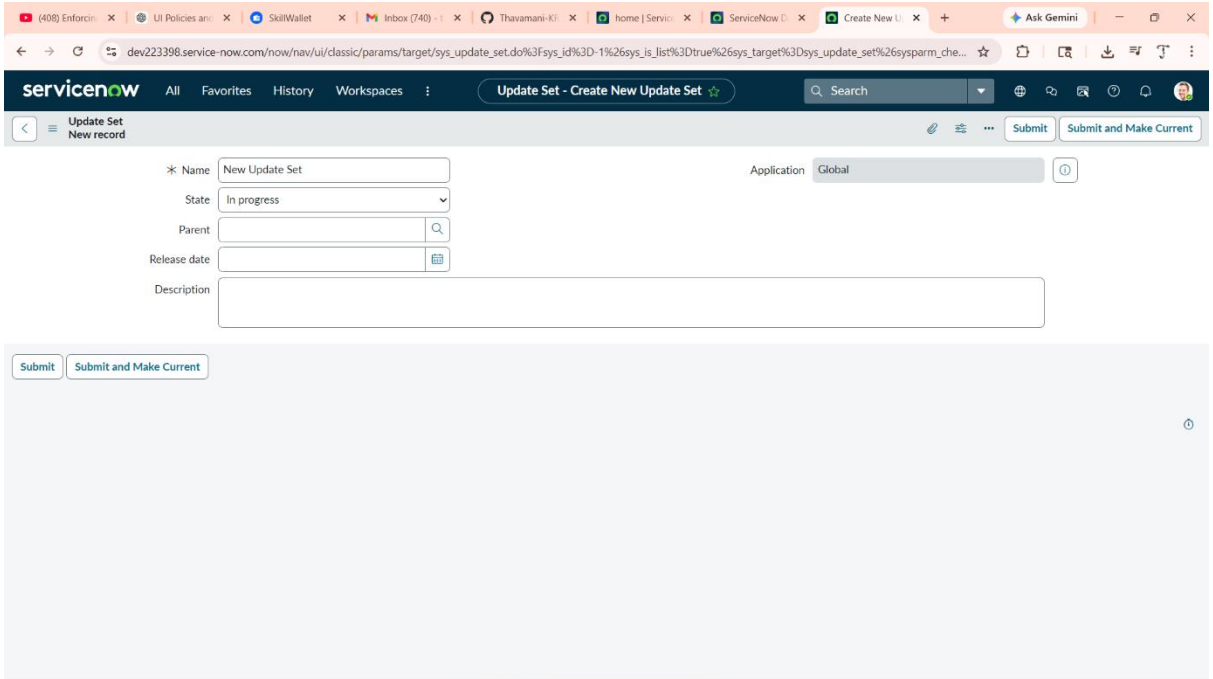
Update Sets are containers used to capture and transfer configuration changes from one instance to another (e.g., Development to Production).

4. Detailed Execution of Project Phases

4.1 Create Update Set

1. Navigate to **System Update Sets → Local Update Sets**.
2. Click on **New**.
3. Enter the following details:
   - Name: UI Policy Project
   - State: In Progress
4. Click **Submit**.
5. Open the created Update Set and click **Make Current**.

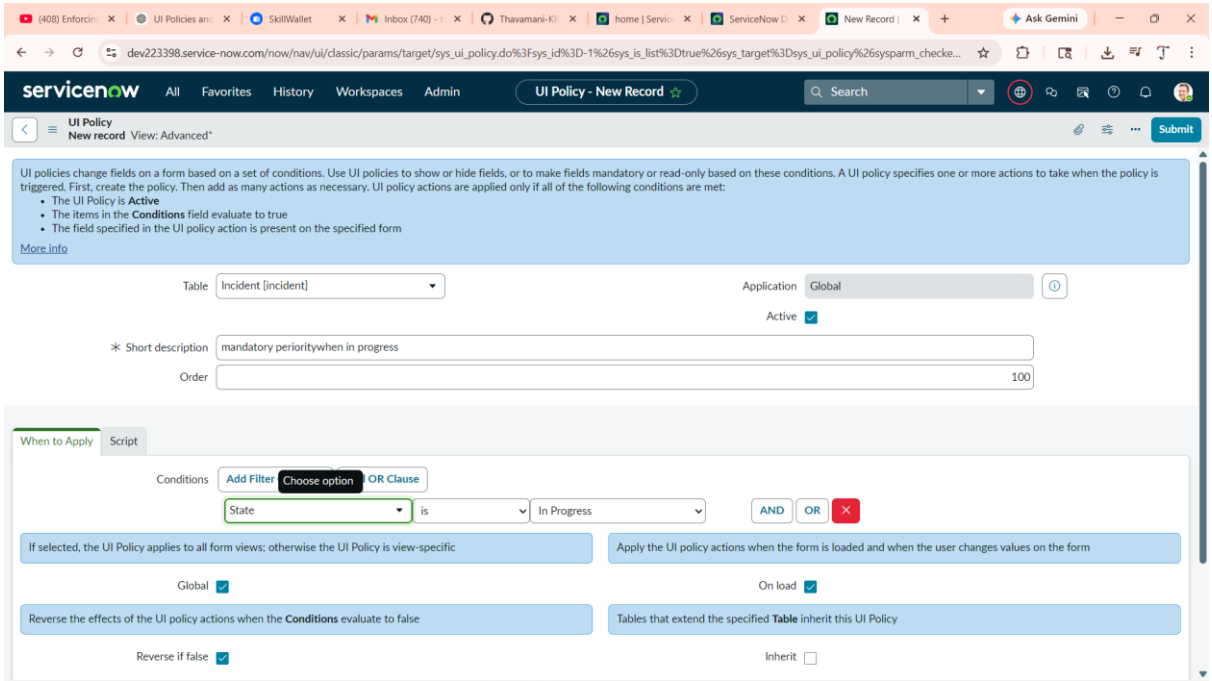
👉 This ensures all changes are captured in this Update Set.



4.2 Create UI Policy

1. Navigate to **System UI → UI Policies**.
2. Click **New**.

3. Enter:
    - Name: Make Description Mandatory
    - Table: Incident
4. Set condition:
    - Priority = High
5. Check **Reverse if false** option.
6. Click **Submit**.

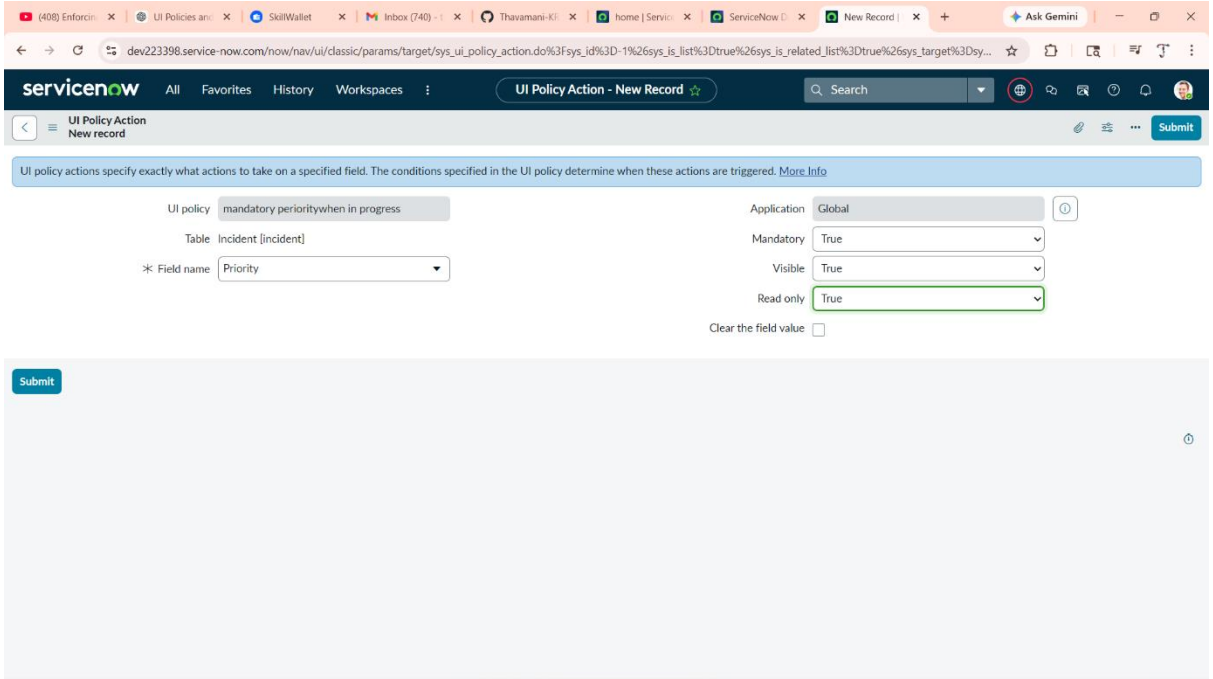


4.3 Create UI Policy Action

1. Open the created UI Policy.
2. Scroll to **UI Policy Actions** related list.
3. Click **New**.

4. Fill details:
    - Field Name: Description
    - Mandatory: True
5. Click **Submit**.

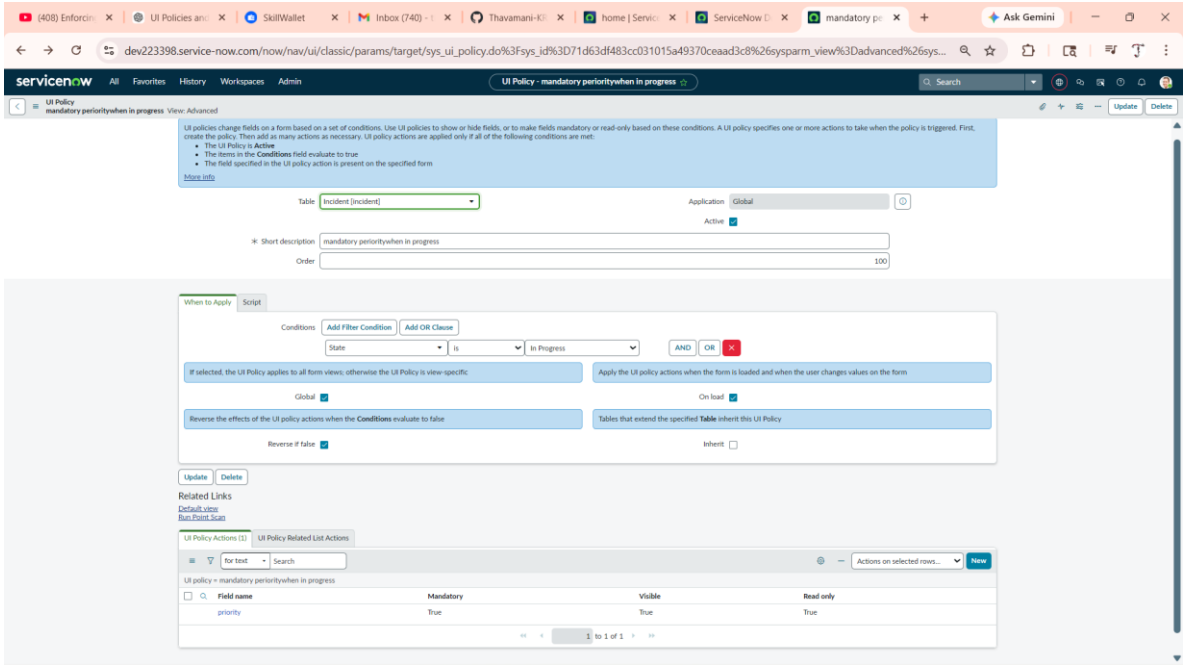
👉 Now the Description field becomes mandatory when Priority is High.



4.4 Check Update Set

1. Navigate to **System Update Sets** → **Local Update Sets**.
2. Open your Update Set.
3. Verify that the UI Policy and related changes are captured.

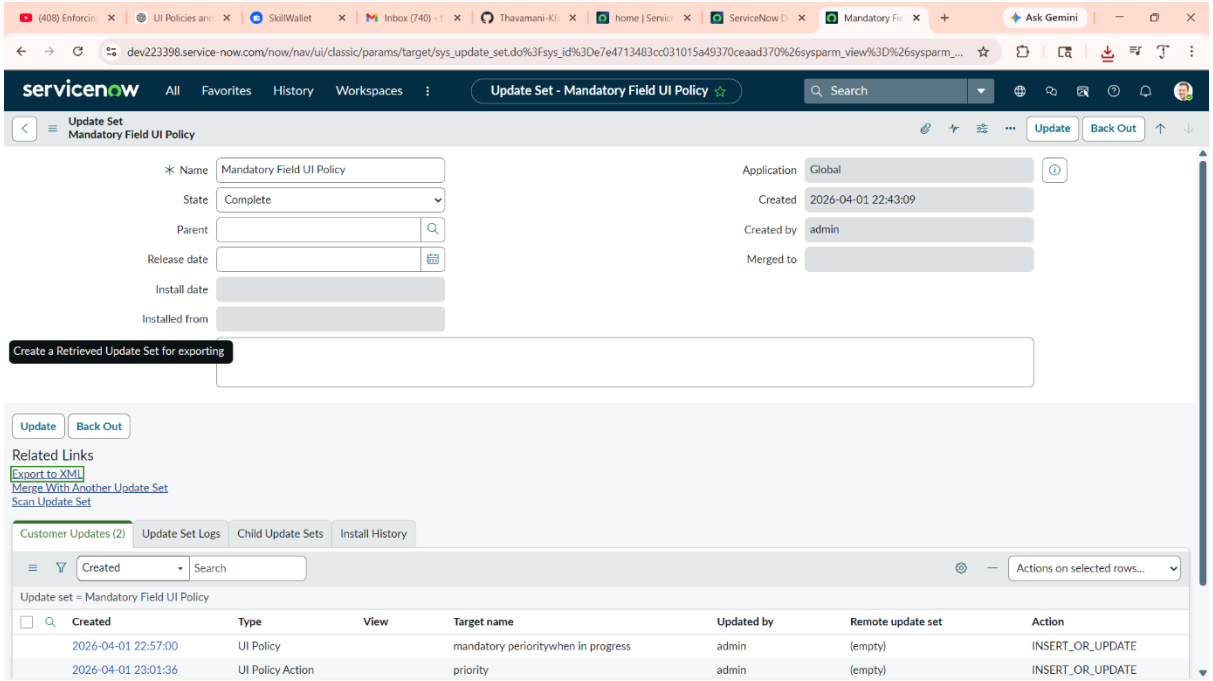
4. You can view captured changes under **Customer Updates**.



4.5 Export and Import the Update Set

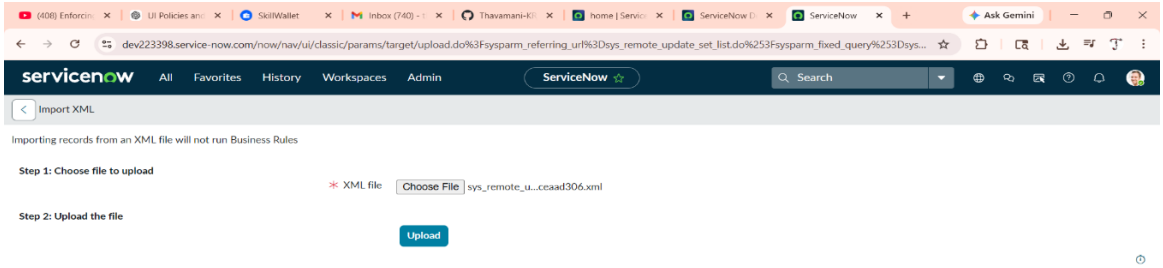
Export

1. Change Update Set state to **Complete**.
2. Click **Export to XML**.
3. Save the XML file.



Import

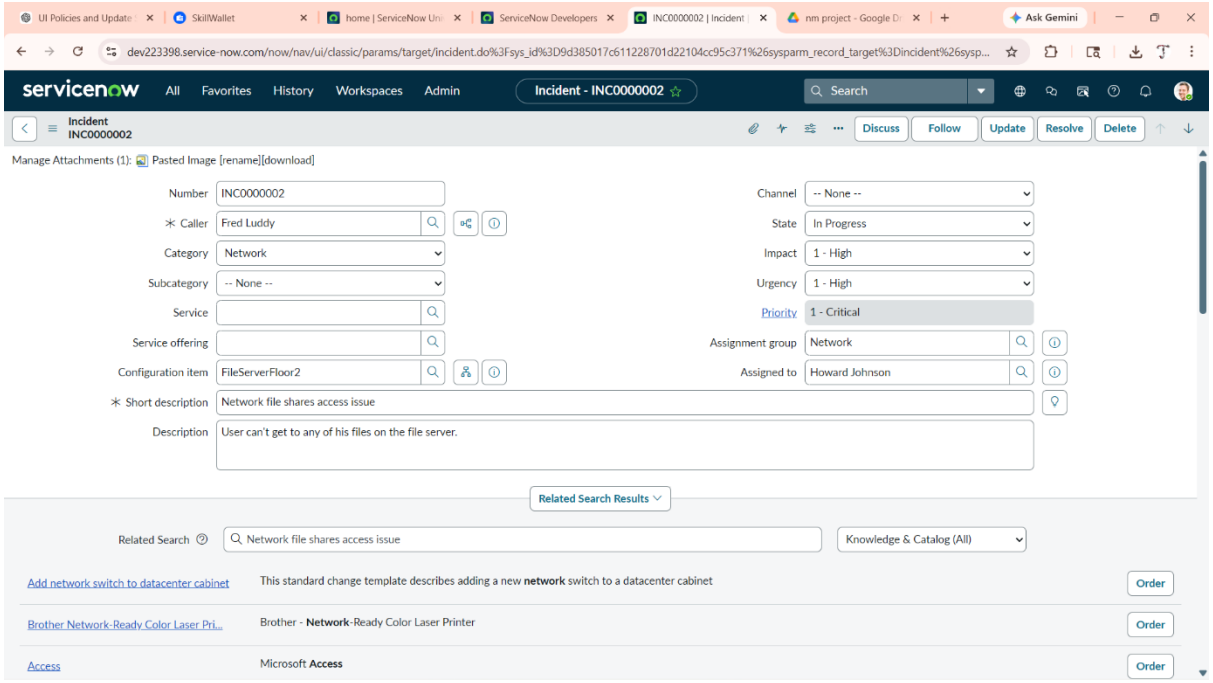
1. Navigate to **System Update Sets → Retrieved Update Sets**.
2. Click **Import Update Set from XML**.
3. Upload the XML file.
4. Click **Preview** to check for errors.
5. Click **Commit** to apply changes.



4.6 Testing Incident

1. Navigate to **Incident** → **Create New**.
2. Set Priority to **High**.
3. Observe that:
    - Description field becomes mandatory.
4. Try submitting without Description:
    - System will show error.
5. Enter Description and submit successfully.

👉 This confirms that the UI Policy is working correctly.



5. Conclusion

This project successfully demonstrates how UI Policies can be used to enforce mandatory fields dynamically, improving data quality and consistency. It also highlights the importance of Update Sets in managing and migrating changes across ServiceNow instances. By implementing these concepts, organizations can ensure better control over configurations and maintain a structured deployment process.

6. Future Enhancements (Optional)

- Implement multiple UI Policies with complex conditions.
- Use Client Scripts for advanced validations.

- Automate deployment using CI/CD integration.